

GPU 클러스터 구조와 모델 병렬화 전략에 따른
LLM 분산 추론 성능 비교

2026/6/6

학번: 202220735

학과: 소프트웨어학과

이름: 정서영

목 차

1. 개요
2. 배경
 - 2.1. 모델 병렬화 (Model parallelism)
 - 2.2. 텐서 병렬화 (Tensor parallelism)
 - 2.3. 파이프라인 병렬화 (Pipeline parallelism)
 - 2.4. ASTRA-sim
3. 프로젝트 소개 및 목적
4. 실험
 - 4.1. 실험 가정
 - 4.2. 실험 결과 및 분석
5. 결론
6. 생성형 AI 사용 부분
7. 참고 자료

요약

본 프로젝트는 GPU 클러스터 구조와 모델 병렬화 전략에 따른 LLM 분산 추론 성능을 비교하는 시뮬레이션 기반 연구이다. ASTRA-sim 을 활용하여 LLaMA-like transformer 모델을 가정하고, 총 8 개의 GPU 를 1 node x 8 GPUs, 2 nodes x 4 GPUs, 4 nodes x 2 GPUs, 8 nodes x 1 GPU 와 같이 서로 다른 클러스터 구조로 배치하였다. 각 환경에서 TP8, PP8, TP4PP2, TP2PP4 등의 병렬화 전략을 적용하고, 전체 workload 처리 완료 시간과 단위 시간당 microbatch 처리량을 기준으로 성능을 비교하였다.

실험에서는 노드 내부 연결로 NVLink 와 PCIe 를 각각 가정하였고, 노드 간 통신은 별도의 inter-node network 로 설정하였다. Tensor parallelism 은 layer 내부 연산을 여러 GPU 에 나누어 수행하는 방식으로, AllReduce 통신 비용의 영향을 크게 받는다고 보았다. 반면 pipeline parallelism 은 모델의 layer 를 여러 stage 로 나누어 처리하고, stage 간 activation 을 전달하는 방식으로 가정하였다. 실험 결과, NVLink 처럼 노드 내부 통신 성능이 높은 환경에서는 TP 기반 전략이 전반적으로 유리하게 나타났다. 반면 PCIe 환경이나 여러 노드로 GPU 가 분산된 환경에서는 TP 의 collective communication 비용이 커져 PP 또는 TP 와 PP 를 결합한 hybrid 전략이 더 좋은 성능을 보이는 경우가 있었다. 이를 통해 LLM 분산 추론 성능은 단순히 GPU 수에 의해 결정되는 것이 아니라, GPU 배치, 네트워크 구조, 병렬화 방식의 조합에 따라 달라진다는 점을 확인하였다.

1. 개요

본 프로젝트는 GPU 클러스터 환경에서 LLM 분산 추론을 수행할 때, 병렬화 방식과 GPU 간 네트워크 구조가 추론 성능에 어떤 영향을 미치는지 분석하는 것을 목표로 한다. LLM 은 모델 크기와 요청량이 증가하면서 단일 GPU 만으로 처리하기 어려워지고 있으며, 이를 해결하기 위해 여러 GPU 에 모델과 연산을 분산하는 tensor parallelism(TP), pipeline parallelism(PP) 등의 병렬화 전략이 사용된다. 이를 위해 ASTRA-sim 을 활용하여 LLaMA-like transformer 모델을 가정하고, 1 node x 8 GPUs, 2 nodes x 4 GPUs, 4 nodes x 2 GPUs, 8 nodes x 1 GPU 환경에서 TP8, PP8, TP4PP2, TP2PP4 등의 구성을 비교하였다. 또한 노드 내부 GPU 연결이 NVLink 인 경우와 PCIe 인 경우를 나누어, 네트워크 대역폭과 지연 시간이 전체 추론 시간과 throughput 에 미치는 영향을 측정하였다. 성능 지표로는 전체 workload 가 완료될 때까지 걸리는 시간과 초당 처리 가능한 microbatch 수인 throughput 을 사용하였다.

2. 배경

2.1. 모델 병렬화 (Model Parallelism)

최근 LLM 은 파라미터 수와 연산량이 크게 증가하면서 단일 GPU 만으로 모델을 저장하거나 추론을 수행하기 어려워지고 있다. 특히 모델 크기가 GPU 메모리 용량을 초과하거나, 하나의

GPU 에서 모든 연산을 처리할 경우 추론 지연 시간이 커지는 문제가 발생한다. 이러한 한계를 해결하기 위해 모델의 연산과 파라미터를 여러 GPU 에 나누어 배치하는 모델 병렬화가 사용된다. 모델 병렬화는 하나의 모델을 여러 GPU 가 함께 처리하도록 만드는 방식으로, 큰 모델을 여러 장치에 분산하여 실행할 수 있다는 장점이 있다. 대표적인 방식으로는 하나의 레이어 내부 연산을 여러 GPU 가 나누어 수행하는 텐서 병렬화와, 모델의 여러 레이어를 GPU 별 stage 로 나누어 순차적으로 처리하는 파이프라인 병렬화가 있다. 본 프로젝트에서는 이러한 병렬화 방식이 GPU 배치 및 네트워크 구조에 따라 LLM 추론 성능에 어떤 차이를 만드는지 분석한다.

2.2. 텐서 병렬화 (Tensor Parallelism)

텐서 병렬화는 하나의 레이어 안에서 수행되는 큰 행렬 연산이나 tensor 를 여러 GPU 에 나누어 동시에 계산하는 방식이다. 예를 들어 transformer 의 attention 또는 MLP 연산을 여러 GPU 가 분할하여 처리하면, 각 GPU 가 담당하는 계산량을 줄일 수 있다. 따라서 GPU 간 연결이 빠른 환경에서는 한 레이어의 연산을 병렬로 처리할 수 있어 추론 시간을 줄이는 데 유리하다. 하지만 텐서 병렬화는 각 GPU 가 계산한 중간 결과를 다시 합치는 과정이 필요하다. 이때 주로 AllReduce 와 같은 collective communication 이 발생하며, 이 통신 비용은 GPU 간 네트워크 성능에 크게 영향을 받는다. 따라서 NVLink 처럼 대역폭이 높고 지연 시간이 낮은 연결에서는 텐서 병렬화의 장점이 잘 드러나지만, 노드 간 네트워크나 PCIe 처럼 상대적으로 느린 통신 환경에서는 AllReduce 비용이 커져 성능이 저하될 수 있다.

2.3. 파이프라인 병렬화 (Pipeline Parallelism)

파이프라인 병렬화는 모델의 레이어를 여러 구간으로 나누고, 각 구간을 서로 다른 GPU 또는 GPU 그룹에 배치하는 방식이다. 각 stage 는 자신이 담당한 레이어들을 순차적으로 처리하고, 계산이 끝난 activation 을 다음 stage 로 전달한다. 이 방식은 하나의 레이어 내부를 나누는 텐서 병렬화와 달리, 모델의 깊이 방향으로 연산을 분산한다는 특징이 있다.

파이프라인 병렬화는 GPU 간에 모든 중간 결과를 자주 합치는 대신, stage 사이에서 activation 을 전달하는 방식으로 동작한다. 따라서 텐서 병렬화처럼 매 레이어마다 AllReduce 가 발생하는 상황을 줄일 수 있으며, 특히 여러 노드로 GPU 가 분산된 환경에서 노드 간 AllReduce 비용을 피하는 데 유리할 수 있다. 다만 stage 들이 순차적으로 연결되어 있기 때문에 microbatch 수가 적을 경우 pipeline bubble 로 인해 GPU 활용률이 낮아질 수 있다. 반대로 microbatch 수가 증가하면 pipeline 이 더 잘 채워져 throughput 이 향상될 수 있다.

2.4. ASTRA-sim

ASTRA-sim 은 대규모 병렬 시스템에서 분산 딥러닝 workload 의 실행 성능을 분석하기 위한 시뮬레이션 도구이다. 실제 GPU 클러스터를 직접 구축하지 않아도, 연산량, 통신량, 병렬화 방식,

네트워크 구조 등을 설정하여 다양한 시스템 환경에서의 성능을 예측할 수 있다. 이를 통해 특정 병렬화 전략이 어떤 하드웨어 구성에서 유리한지 사전에 비교할 수 있다는 장점이 있다.

분산 딥러닝에서는 계산 시간뿐만 아니라 GPU 간 통신 시간이 전체 성능에 큰 영향을 미친다.

특히 tensor parallelism에서는 collective communication 이 자주 발생하고, pipeline parallelism에서는 stage 간 activation 전달이 필요하다. ASTRA-sim 은 이러한 연산과 통신의 흐름을 함께 고려하여 추론 시간과 throughput 을 분석할 수 있도록 도와준다.

따라서 ASTRA-sim 은 실제 클러스터 환경에서 모든 조합을 실험하기 어려운 경우, 여러 병렬화 전략과 네트워크 구성을 비교하기 위한 유용한 도구로 활용될 수 있다. 본 프로젝트에서도 GPU 간 통신 구조와 병렬화 방식에 따른 LLM 추론 성능 차이를 분석하기 위해 ASTRA-sim 을 사용하였다.

3. 프로젝트 소개 및 목적

본 프로젝트는 GPU 클러스터 환경에서 LLM 분산 추론을 수행할 때, 병렬화 방식과 GPU 간 네트워크 구조에 따라 추론 시간과 throughput 이 어떻게 달라지는지를 비교·분석하는 시뮬레이션 기반 프로젝트이다. 최근 LLM 은 모델 크기와 연산량이 커지면서 단일 GPU 만으로 추론을 수행하기 어려워지고 있으며, 이를 해결하기 위해 여러 GPU 에 모델과 연산을 나누어 배치하는 병렬화 전략이 사용된다. 그러나 병렬화 전략을 적용한다고 해서 항상 성능이 향상되는 것은 아니며, GPU 간 통신 방식과 네트워크 구조에 따라 오히려 통신 비용이 병목이 될 수 있다.

본 프로젝트에서는 이러한 문제의식을 바탕으로 tensor parallelism(TP), pipeline parallelism(PP), 그리고 TP 와 PP 를 결합한 hybrid parallelism 전략을 비교한다. TP 는 하나의 레이어 내부 연산을 여러 GPU 가 나누어 동시에 계산하는 방식으로, 계산량을 줄일 수 있지만 AllReduce 와 같은 collective communication 이 자주 발생한다. 반면 PP 는 모델의 레이어를 여러 stage 로 나누어 각 GPU 또는 GPU 그룹에 배치하는 방식으로, stage 간 activation 전달이 필요하지만 TP 에 비해 반복적인 AllReduce 비용을 줄일 수 있다. 따라서 두 방식은 계산 분산 방식뿐만 아니라 통신 패턴에서도 차이가 있으며, 어떤 방식이 유리한지는 GPU 가 어떻게 배치되어 있는지와 네트워크 성능에 따라 달라질 수 있다.

이를 분석하기 위해 본 프로젝트는 실제 GPU 클러스터를 직접 구축하는 대신 ASTRA-sim 을 활용한 시뮬레이션 방식으로 실험을 수행한다. LLaMA-like transformer 모델을 가정하고, 1 node x 8 GPUs, 2 nodes x 4 GPUs, 4 nodes x 2 GPUs, 8 nodes x 1 GPU 와 같은 다양한 클러스터 구성을 설정한다. 또한 TP8, PP8, TP4PP2, TP2PP4 등의 병렬화 전략을 적용하여, 동일한 GPU 수를 사용하더라도 GPU 배치와 병렬화 방식에 따라 성능이 어떻게 달라지는지 비교한다.

특히 본 프로젝트는 노드 내부 GPU 간 연결이 NVLink 인 경우와 PCIe 인 경우를 구분하여 분석한다. NVLink 는 높은 대역폭과 낮은 추론 시간을 제공하므로 TP 에서 발생하는 AllReduce 비용을 줄이는데 유리할 수 있다. 반면 PCIe 또는 노드 간 네트워크처럼 상대적으로 느린 연결에서는 TP 의

collective communication 비용이 커질 수 있으며, 이 경우 PP 또는 TP와 PP를 결합한 구성이 더 효율적일 수 있다. 따라서 본 프로젝트는 단순히 TP와 PP 중 어느 방식이 더 좋은지를 판단하는 것이 아니라, 네트워크 구조와 통신 패턴을 함께 고려했을 때 어떤 병렬화 전략이 더 적합한지를 비교하는데 목적이 있다.

성능 평가는 전체 workload가 완료될 때까지 걸리는 시간과 초당 처리 가능한 microbatch 수를 의미하는 throughput을 기준으로 진행한다. 이를 통해 각 병렬화 전략이 지연 시간 측면에서 유리한지, 또는 처리량 측면에서 유리한지를 함께 확인한다. 최종적으로 본 프로젝트는 LLM 분산 추론에서 성능을 결정하는 핵심 요소가 GPU 수뿐만 아니라 병렬화 방식, GPU 배치, 네트워크 구조, 통신 비용의 조합임을 시뮬레이션 결과를 통해 확인하고자 한다.

4. 실험

4.1. 실험 가정

본 프로젝트는 실제 GPU 클러스터를 직접 구성하여 실험하는 대신, ASTRA-sim을 활용한 시뮬레이션 기반 실험으로 진행하였다. 실험에서는 LLaMA 계열 transformer 모델을 가정하고, 동일한 모델과 workload 조건에서 GPU 클러스터 구조와 모델 병렬화 전략에 따른 성능 차이를 비교하였다. 이를 통해 특정 병렬화 방식이 어떤 네트워크 환경과 GPU 배치에서 더 유리한지 분석하고자 하였다.

모델은 80개의 layer를 가지는 LLaMA-like transformer로 설정하였다. hidden size는 8192, intermediate size는 28672, sequence length는 2048로 가정하였으며, microbatch당 batch size는 1로 두었다. activation dtype은 FP16/BF16을 기준으로 하였고, 원소 하나의 크기는 2 byte로 계산하였다. 이에 따라 한 번 전달되는 activation 크기는 32 MiB로 설정하였다.

연산 시간은 attention, MLP, 전체 layer 단위로 나누어 가정하였다. 하나의 full attention 연산은 1237 us, full MLP 연산은 2887 us, full layer 연산은 4124 us로 설정하였다. Tensor parallelism을 적용하는 경우에는 layer 내부 연산이 TP degree만큼 나누어 수행된다고 가정하였다. 반면 pipeline parallelism에서는 각 stage가 자신에게 할당된 layer를 full compute로 처리한다고 보았다. 예를 들어 PP8의 경우 전체 80개 layer를 8개 stage로 나누어, 각 stage가 10개 layer를 담당하도록 설정하였다.

네트워크 환경은 노드 내부 연결과 노드 간 연결을 구분하여 가정하였다. 노드 내부 연결은 NVLink와 PCIe 환경을 각각 고려하였으며, NVLink는 400 GB/s의 bandwidth와 0.93625 us의 latency를 가지는 것으로 설정하였다. PCIe는 32 GB/s의 bandwidth와 1.500 us의 latency를

가지는 것으로 설정하였다. 노드 간 네트워크는 25 GB/s 의 bandwidth 와 5.000 us 의 latency 를 가지는 것으로 가정하였다.

실험에서는 총 8 개의 GPU 를 사용하는 여러 클러스터 구조를 비교하였다. 구체적으로 1 node x 8 GPUs, 2 nodes x 4 GPUs, 4 nodes x 2 GPUs, 8 nodes x 1 GPU 구성을 사용하였다. 각 구성에서 TP8, PP8, TP4PP2, TP2PP4 와 같은 병렬화 전략을 적용하여 성능을 비교하였다. 여기서 TP 는 Tensor Parallelism, PP 는 Pipeline Parallelism 을 의미하며, 뒤에 붙은 숫자는 해당 병렬화 방식에 사용되는 GPU 수 또는 stage 수를 나타낸다.

구체적으로, TP8 은 8 개의 GPU 전체를 하나의 tensor parallel group 으로 묶는 방식이다. 즉, 하나의 layer 내부에서 발생하는 attention 이나 MLP 연산을 8 개의 GPU 가 나누어 동시에 계산한다. 각 GPU 는 전체 연산의 일부만 담당하므로 계산량은 줄어들지만, 계산 결과를 합치기 위해 AllReduce 와 같은 GPU 간 collective communication 이 발생한다.

PP8 은 8 개의 GPU 를 8 개의 pipeline stage 로 나누는 방식이다. 전체 모델의 layer 를 8 개 구간으로 나누고, 각 GPU 가 하나의 stage 를 담당한다. 예를 들어 80 개의 layer 를 가진 모델이라면 각 stage 는 약 10 개의 layer 를 담당한다. 이 방식에서는 각 stage 가 자신의 layer 연산을 수행한 뒤, 다음 stage 로 activation 을 전달한다.

TP4PP2 는 tensor parallelism 과 pipeline parallelism 을 함께 사용하는 hybrid parallelism 방식이다. 8 개의 GPU 를 4 개씩 묶어 두 개의 pipeline stage 를 구성한다. 각 stage 안에서는 4 개의 GPU 가 tensor parallelism 으로 layer 내부 연산을 나누어 계산하고, stage 사이에서는 pipeline parallelism 처럼 activation 을 전달한다. 따라서 TP4PP2 는 노드 내부 GPU 들을 TP group 으로 묶고, 노드 간에는 PP 방식으로 연결하는 형태로 볼 수 있다.

TP2PP4 는 8 개의 GPU 를 2 개씩 묶어 네 개의 pipeline stage 를 구성하는 방식이다. 각 stage 안에서는 2 개의 GPU 가 tensor parallelism 으로 연산을 나누어 수행하고, 총 4 개의 stage 가 pipeline 방식으로 연결된다. TP4PP2 에 비해 각 TP group 의 크기는 작지만 pipeline stage 수는 더 많아지므로, 계산 분산 정도와 stage 간 통신 패턴이 달라진다.

정리하면, TP8 은 layer 내부 연산을 최대한 많이 나누는 전략이고, PP8 은 모델의 layer 를 stage 단위로 나누는 전략이다. TP4PP2 와 TP2PP4 는 TP 와 PP 를 결합한 방식으로, GPU 를 몇 개씩 묶어 tensor parallel group 을 만들고 이를 몇 개의 pipeline stage 로 연결하는지에 따라 구분된다.

성능 지표로는 batch_time_ms 와 throughput_mb_s 를 사용하였다. batch_time_ms 는 단일 요청의 응답 시간이 아니라, 주어진 microbatch 들로 구성된 전체 workload 가 모두 처리되기까지 걸린 시간을 의미한다. throughput_mb_s 는 단위 시간당 처리 가능한 microbatch 수를 의미한다.

Microbatch 는 하나의 큰 batch 를 더 작은 단위로 나눈 것을 의미한다. 일반적인 batch 가 여러 요청이나 입력 데이터를 한 번에 묶어 처리하는 단위라면, microbatch 는 이 batch 를 더 잘게 나누어 순차적으로 또는 pipeline 형태로 처리하기 위한 단위이다. 본 프로젝트에서는 microbatch 당 batch size 를 1로 가정하고, microbatch 수를 1, 2, 4, 8로 변화시키며 성능을 비교하였다.

Microbatch 는 특히 pipeline parallelism 에서 중요한 역할을 한다. Pipeline parallelism 은 모델의 layer 를 여러 stage 로 나누고 각 stage 가 순차적으로 연산을 수행하는 방식이다. 이때 입력이 하나의 큰 batch 로만 들어오면, 앞 stage 가 계산을 끝내고 다음 stage 로 넘기는 동안 일부 GPU 가 대기하게 되어 pipeline bubble 이 발생할 수 있다. 반면 batch 를 여러 microbatch 로 나누면, 첫 번째 microbatch 가 다음 stage 로 이동한 뒤에도 이전 stage 는 두 번째 microbatch 를 처리할 수 있으므로 여러 stage 가 동시에 더 효율적으로 동작할 수 있다. 따라서 microbatch 를 도입하면 pipeline 의 활용률을 높이고 처리량을 증가시킬 수 있다. 다만 microbatch 수가 증가한다고 항상 모든 전략에서 성능이 좋아지는 것은 아니다. Tensor parallelism 의 경우 microbatch 수가 증가하면 전체 workload 처리 시간이 증가할 수 있으며, pipeline parallelism 에서는 microbatch 수가 늘어날수록 pipeline 이 더 잘 채워져 처리량이 향상되는 경향이 나타난다. 본 프로젝트에서는 이러한 차이를 확인하기 위해 microbatch 수를 변화시키며 batch time 과 처리량을 측정하였다.

4.2. 실험 결과 및 분석

먼저 NVLink 환경에서의 실험 결과 및 분석이다.

1 node x 8 GPUs 환경에서는 모든 GPU 가 하나의 노드 안에 위치하므로, GPU 간 통신이 모두 노드 내부에서 발생한다. 이 경우 NVLink 의 높은 대역폭과 낮은 지연 시간 덕분에 TP8 의 AllReduce 비용이 상대적으로 작게 나타나며, TP8 이 PP8 보다 낮은 batch time 과 높은 처리량을 보였다.

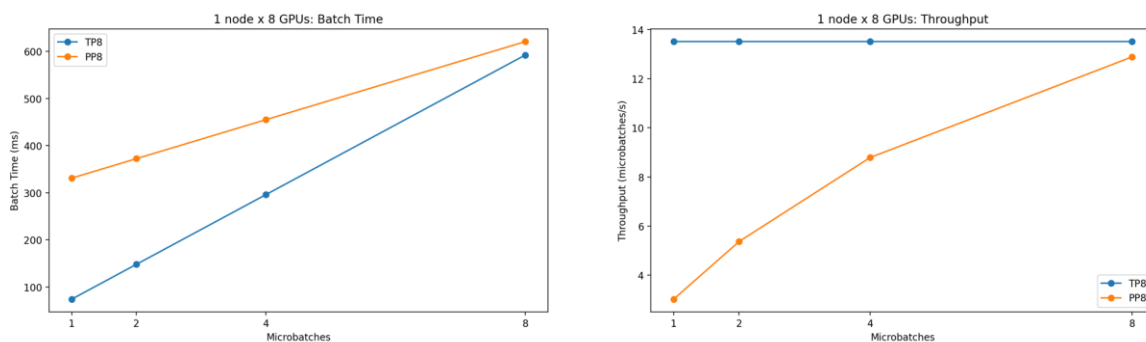


그림 1. NVLink 환경에서 모델 병렬화 전략에 따른 1 node x 8 GPU 조합 Batch Time(좌)과 Throughput (우)

2 nodes x 4 GPUs 환경에서는 TP8 과 TP4PP2 가 비교적 좋은 성능을 보였다. TP8 은 전체 GPU 를 사용해 layer 내부 연산을 나누기 때문에 batch time 이 낮게 나타났고, TP4PP2 는 노드 내부에서는 TP 를 사용하고 노드 간에는 PP 형태로 연결되므로 microbatch 수가 증가할수록 처리량 측면에서 장점이 나타났다.

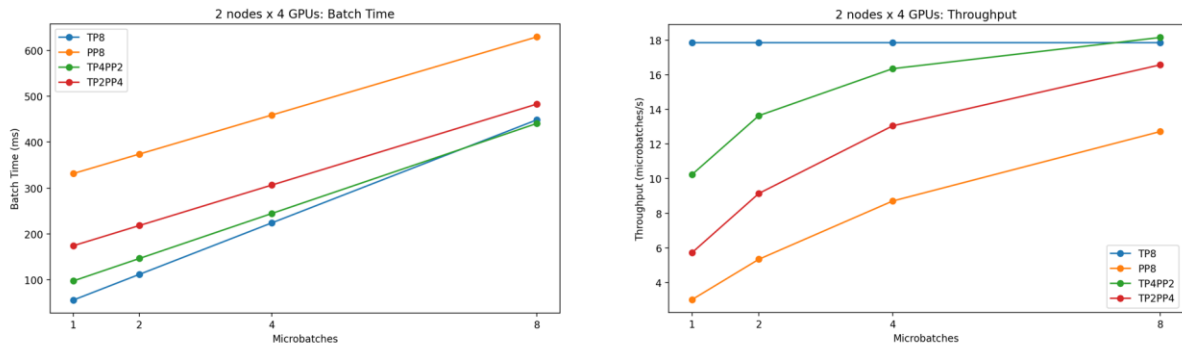


그림 2. NVLink 환경에서 모델 병렬화 전략에 따른 2 nodes x 4 GPUs 조합 Batch Time(좌)과 Throughput (우)

4 nodes x 2 GPUs 환경에서도 TP8 이 가장 낮은 batch time 과 높은 처리량을 보였다. 다만 TP4PP2 도 TP2PP4 보다 안정적으로 좋은 성능을 보였으며, 이는 각 stage 의 계산량과 통신 구조가 성능에 영향을 준다는 점을 보여준다.

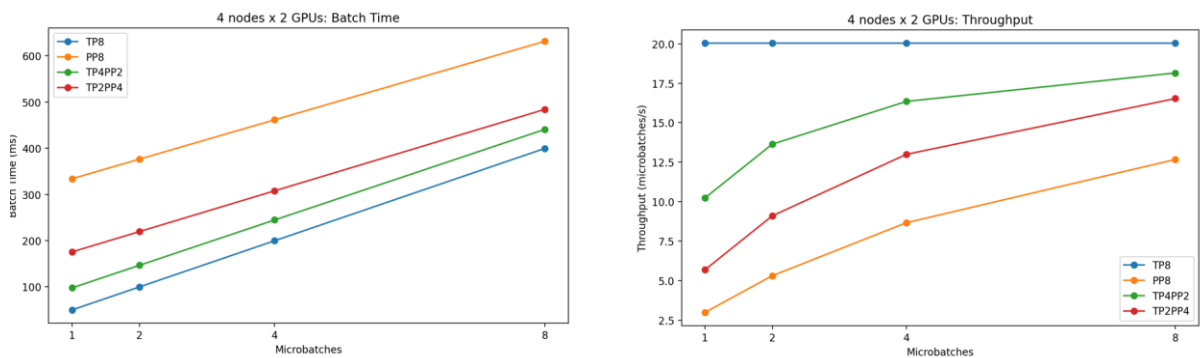


그림 3. NVLink 환경에서 모델 병렬화 전략에 따른 4 nodes x 2 GPUs 조합 Batch Time(좌)과 Throughput (우)

8 nodes x 1 GPU 환경에서는 각 노드에 GPU 가 하나씩만 존재하므로 TP8 에서 발생하는 AllReduce 가 모두 노드 간 통신으로 수행된다. 이로 인해 TP8 의 batch time 이 크게 증가하고 처리량이 낮게 유지되며, 반대로 PP8 은 microbatch 수가 증가할수록 pipeline 활용도가 높아져 더 유리한 성능을 보였다.

GPU 클러스터 구조와 모델 병렬화 전략에 따른 LLM 분산 추론 성능 비교

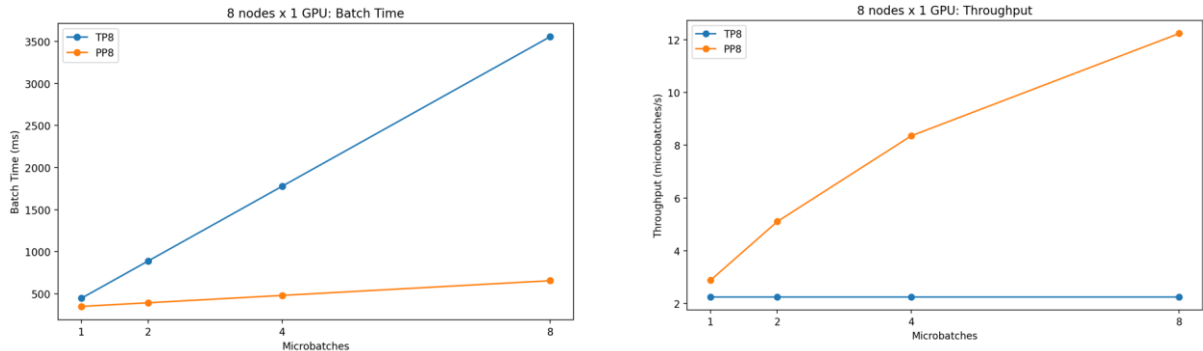


그림 4. NVLink 환경에서 모델 병렬화 전략에 따른 8 nodes x 1 GPU 조합 Batch Time(좌)과 Throughput (우)

다음은 PCIe 환경에서의 결과 및 분석이다.

PCIe 환경의 1 node x 8 GPUs에서는 TP8의 성능이 NVLink 환경보다 크게 저하되었다. PCIe는 NVLink보다 대역폭이 낮기 때문에 TP8에서 반복적으로 발생하는 AllReduce 비용이 커졌고, microbatch 수가 증가할수록 PP8이 batch time과 처리량 측면에서 더 유리하게 나타났다.

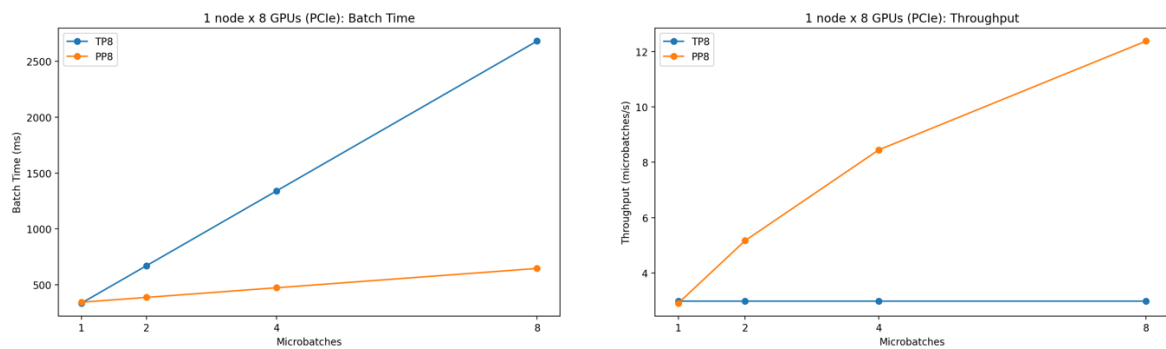


그림 5. PCIe 환경에서 모델 병렬화 전략에 따른 1 node x 8 GPUs 조합 Batch Time(좌)과 Throughput (우)

2 nodes x 4 GPUs의 PCIe 환경에서는 TP8이 작은 microbatch에서는 낮은 batch time을 보였지만, microbatch 수가 증가하면 PP8 또는 TP2PP4가 더 좋은 처리량을 보였다. 이는 PCIe와 노드 간 네트워크가 섞인 환경에서는 TP의 collective communication 비용이 커져, PP 또는 hybrid 전략이 더 유리해질 수 있음을 보여준다.

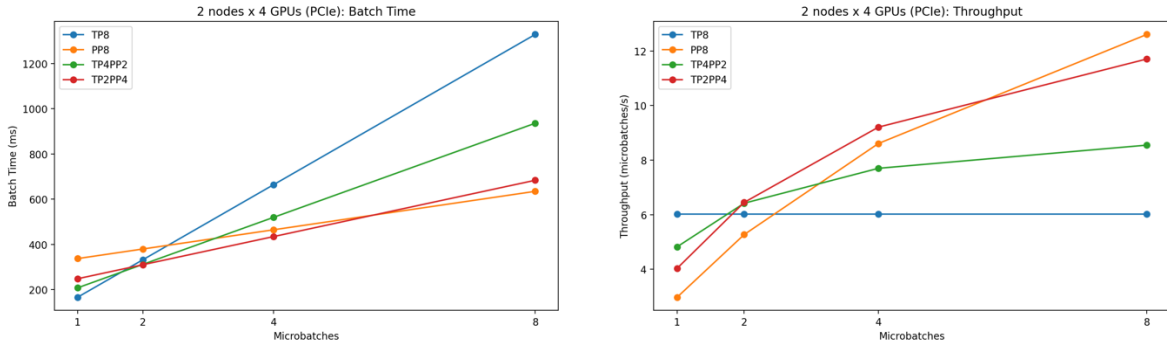


그림 6. PCIe 환경에서 모델 병렬화 전략에 따른 2 nodes x 4 GPUs 조합 Batch Time(좌)과 Throughput (우)

4 nodes x 2 GPUs의 PCIe 환경에서도 microbatch 수가 작을 때는 TP8의 batch time이 낮게 나타난다. 그러나 microbatch 수가 증가할수록 PP8과 TP2PP4의 처리량이 더 높아지며, 특히 TP2PP4는 TP4PP2보다 더 좋은 성능을 보였다. 이는 노드 수가 증가하고 통신 경로가 복잡해질수록 TP degree를 크게 잡는 방식이 항상 유리하지 않다는 점을 보여준다.

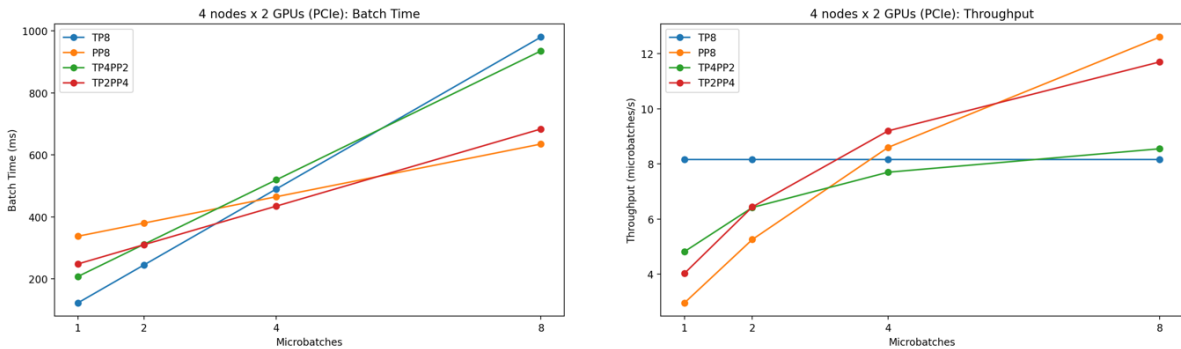


그림 7. PCIe 환경에서 모델 병렬화 전략에 따른 4 nodes x 2 GPUs 조합 Batch Time(좌)과 Throughput (우)

8 nodes x 1 GPU 구성에서는 노드 내부 GPU 연결이 없으므로 NVLink와 PCIe의 구분이 직접적으로 적용되지 않는다. 이 환경에서는 모든 GPU 간 통신이 노드 간 네트워크를 통해 발생하며, TP8은 AllReduce 비용이 크게 증가하여 성능이 저하된다. 반면 PP8은 stage 간 activation 전달만 수행하므로 microbatch 수가 증가할수록 더 높은 처리량을 보였다.

5. 결론

본 프로젝트에서는 ASTRA-sim을 활용하여 GPU 클러스터 구조와 모델 병렬화 전략에 따른 LLM 분산 추론 성능을 비교하였다. 실험에서는 동일한 LLaMA-like transformer 모델을 가정하고, 총 8개의 GPU를 1 node x 8 GPUs, 2 nodes x 4 GPUs, 4 nodes x 2 GPUs, 8 nodes x 1 GPU와 같이

서로 다른 클러스터 구조로 배치하였다. 또한 TP8, PP8, TP4PP2, TP2PP4 등의 병렬화 전략을 적용하여 batch time 과 처리량이 어떻게 달라지는지 분석하였다.

실험 결과, 병렬화 전략의 성능은 단순히 GPU 수에 의해 결정되지 않고, GPU 가 어떤 구조로 배치되어 있는지와 GPU 간 통신 비용에 크게 영향을 받는다는 점을 확인할 수 있었다. NVLink 환경에서는 노드 내부 통신 성능이 높기 때문에 TP 기반 전략이 전반적으로 좋은 성능을 보였다. 특히 1 node x 8 GPUs 처럼 모든 GPU 가 하나의 노드 안에 있는 경우에는 TP8 에서 발생하는 AllReduce 비용이 상대적으로 작아, PP8 보다 낮은 batch time 과 높은 처리량을 보였다.

반면 PCIe 환경에서는 NVLink 보다 낮은 대역폭으로 인해 TP 의 통신 비용이 크게 증가하였다. 특히 microbatch 수가 증가할수록 TP8 의 batch time 이 크게 늘어났고, PP8 또는 TP 와 PP 를 결합한 hybrid 전략이 더 높은 처리량을 보이는 경우가 나타났다. 이는 TP 가 계산을 효과적으로 분산할 수 있는 장점이 있지만, AllReduce 와 같은 collective communication 비용이 커지는 환경에서는 오히려 성능 병목이 될 수 있음을 보여준다.

또한 노드 수가 증가할수록 병렬화 전략의 선택이 더욱 중요해졌다. 8 node x 1 GPU 구성에서는 각 GPU 가 서로 다른 노드에 위치하므로 TP8 의 AllReduce 가 모두 노드 간 통신을 통해 수행된다. 이 경우 TP8 은 batch time 이 크게 증가하고 처리량이 낮게 유지되었으며, PP8 이 더 유리한 성능을 보였다. 즉, 노드 간 통신 비용이 큰 환경에서는 layer 내부 연산을 모든 GPU 에 나누는 TP 방식보다, stage 간 activation 만 전달하는 PP 방식이 더 적합할 수 있다.

종합하면, LLM 분산 추론에서 최적의 병렬화 전략은 하나로 고정될 수 없으며, 클러스터 구조와 네트워크 특성에 따라 달라진다. 노드 내부 통신이 빠른 환경에서는 TP 기반 전략이 유리하지만, 노드 간 통신 비중이 커지거나 PCIe 처럼 상대적으로 느린 연결을 사용하는 경우에는 PP 또는 hybrid 전략이 더 적합할 수 있다. 따라서 효율적인 LLM 추론 시스템을 설계하기 위해서는 모델의 연산량뿐만 아니라, GPU 배치, 네트워크 대역폭, 통신 패턴을 함께 고려해야 한다.

6. 생성형 AI 사용 부분

본 프로젝트에서는 실제 LLM 추론 시스템을 직접 구현한 것이 아니라, ASTRA-sim 을 실행하기 위한 시뮬레이션 설정 파일을 작성하여 TP 와 PP 의 동작을 모사하였다. 설정 파일에는 GPU 클러스터 구성, GPU 간 네트워크 bandwidth 및 latency, 모델의 layer 수와 activation 크기, 병렬화 전략별 GPU 배치 등을 반영하였다.

특히 Tensor Parallelism(TP)의 경우 하나의 layer 내부 연산이 여러 GPU 에 나누어 수행되고, 이후 AllReduce 통신이 발생하는 형태로 가정하였다. Pipeline Parallelism(PP)의 경우 모델의 layer 를 여러 stage 로 나누고, 각 stage 가 담당한 layer 를 처리한 뒤 다음 stage 로 activation 을 전달하는 방식으로 구성하였다. 즉, 실제 TP/PP 프레임워크를 구현한 것이 아니라, ASTRA-sim

입력 파일에서 계산 시간과 통신 시간을 정의하여 TP와 PP의 주요 동작 특성을 흉내 내도록 설계하였다.

해당 시뮬레이션 설정 파일을 작성하는 과정에서는 생성형 AI를 보조적으로 활용하였다.

구체적으로, TP와 PP의 개념 및 동작 방식이 정리된 자료와 실험 환경 정보를 바탕으로, 각 병렬화 전략에서 어떤 연산과 통신이 발생해야 하는지 정리하고 이를 ASTRA-sim에서 사용할 수 있는 형태의 입력 구조로 옮기는 데 도움을 받았다. 다만 최종 실험 조건, GPU 배치, 네트워크 가정, 성능 지표 해석은 프로젝트 목적에 맞게 직접 검토하고 수정하였다.

7. 참고 자료

[1] NVIDIA Developer Blog, "Train Generative AI Models More Efficiently with New NVIDIA Megatron Core Functionalities," NVIDIA, 2024.

Available: <https://developer.nvidia.com/ko-kr/blog/train-generative-ai-models-more-efficiently-with-new-nvidia-megatron-core-functionalities/>

[2] M. Shoeybi, M. Patwary, R. Puri, P. LeGresley, J. Casper, and B. Catanzaro, "Megatron-LM: Training Multi-Billion Parameter Language Models Using Model Parallelism," arXiv:1909.08053, 2019.

Available: <https://arxiv.org/abs/1909.08053>

[3] Y. Huang et al., "GPipe: Efficient Training of Giant Neural Networks using Pipeline Parallelism," arXiv:1811.06965, 2018.

Available: <https://arxiv.org/abs/1811.06965>

[4] ASTRA-sim, "ASTRA-sim: A Distributed Machine Learning System Simulator."

Available: <https://astra-sim.github.io/>

[5] Hugging Face, "Model Parallelism," Transformers Documentation v4.13.0.

Available: <https://huggingface.co/docs/transformers/v4.13.0/parallelism>